

Datenstruktur `ArrayList` in Java

Merkmal	Beschreibung
Dynamische Größe	Eine <code>ArrayList</code> kann ihre Größe dynamisch ändern, wenn Elemente hinzugefügt oder entfernt werden. Sie ist im Gegensatz zu einem <code>Array</code> nicht von einer festen Größe abhängig.
Homogene Datentypen	Alle Elemente in einer <code>ArrayList</code> müssen denselben Datentyp haben, es sei denn, es wird ein generischer Typ verwendet.
Schneller Zugriff	Der Zugriff auf Elemente erfolgt über einen Index (ähnlich wie bei einem <code>Array</code>) und ist in konstanter Zeit, $O(1)$.
Verwaltung von Kapazität	Die <code>ArrayList</code> verwaltet ihre interne Kapazität und wächst automatisch, wenn sie voll ist, um neue Elemente aufzunehmen.
Unterstützung für <code>null</code>	Eine <code>ArrayList</code> kann <code>null</code> -Werte als gültige Elemente enthalten.
Unbegrenzte Kapazität	Die Kapazität einer <code>ArrayList</code> ist theoretisch unbegrenzt und wird nur durch den verfügbaren Speicher des Systems begrenzt.
Thread-Sicherheit	<code>ArrayLists</code> sind nicht thread-sicher. Für Multithread-Anwendungen muss eine explizite Synchronisierung oder die Verwendung von <code>CopyOnWriteArrayList</code> erfolgen.

Methoden

Methode	Beschreibung
<code>add()</code>	Fügt ein Element am Ende der Liste hinzu.
<code>add(index, element)</code>	Fügt ein Element an einer bestimmten Position in der Liste ein.
<code>get()</code>	Gibt das Element an der angegebenen Position zurück.
<code>set()</code>	Ersetzt das Element an einer bestimmten Position durch ein neues Element.
<code>remove()</code>	Entfernt das erste Vorkommen eines bestimmten Elements.
<code>remove(index)</code>	Entfernt das Element an der angegebenen Position.
<code>size()</code>	Gibt die Anzahl der Elemente in der Liste zurück.
<code>isEmpty()</code>	Überprüft, ob die Liste leer ist.
<code>contains()</code>	Überprüft, ob ein bestimmtes Element in der Liste vorhanden ist.
<code>clear()</code>	Entfernt alle Elemente aus der Liste.

Methode	Beschreibung
<code>indexOf()</code>	Gibt den Index des ersten Vorkommens eines bestimmten Elements zurück.
<code>toArray()</code>	Gibt die Array-Darstellung der ArrayList zurück.
<code>subList()</code>	Gibt eine Teilliste der ArrayList zurück, die nur die Elemente im angegebenen Bereich enthält.

Anwendungsszenarien

Szenario	Beschreibung
Dynamische Listen	Eine ArrayList eignet sich besonders für Szenarien, bei denen die Anzahl der Elemente zur Laufzeit verändert wird, da sie sich dynamisch anpasst.
Datenaggregation	Wenn Elemente einer Sammlung (z. B. von Benutzereingaben, Dateien, oder Streams) gesammelt werden müssen, eignet sich eine ArrayList hervorragend für die Speicherung der Daten.
Suchen und Sortieren	Da ArrayLists den schnellen Zugriff auf Elemente ($O(1)$) ermöglichen, sind sie nützlich, wenn nach bestimmten Elementen gesucht oder die Liste sortiert werden muss.
Hinzufügen von Elementen	Eine ArrayList ist besonders nützlich, wenn Elemente häufig hinzugefügt werden, da sie automatisch ihre Kapazität verwaltet.
Verwalten von nicht fixierten Datensätzen	Für Anwendungen, die mit Datensätzen arbeiten, bei denen die Größe zur Laufzeit variiert (z. B. dynamische Formulare, Benutzerdaten), ist die ArrayList eine praktische Wahl.
Verarbeiten von Sammlungen	Wenn mit einer Sammlung von Objekten gearbeitet wird und die Elemente später geändert oder durch Iteration verarbeitet werden sollen, ist die ArrayList eine ideale Wahl.